

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»

Выполнил:

Студент 4 курса

Группы ИИ-26

Яшков П. А.

Проверил:

Никитин С. К.

Цель работы: Реализовать многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с однослойным персептроном, показав принципиальную линейную неразделимость задачи XOR.

Ход работы:

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $E_s \leq E_e$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:
 - количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),
 - итоговая суммарная ошибка, точность (ассигасу) воспроизведения всех 4 примеров таблицы истинности.
6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c_0 или c_1) оно ближе.

Вариант 10 ($c_0 = -3$, $c_1 = -8$)

A	B	$A \oplus B$
-3	-3	-3
-3	-8	-8
-8	-3	-8
-8	-8	-3

Код программы:

```
import numpy as np
```

```
c0, c1 = -3, -8
```

```
X = np.array([
    [-3, -3],
    [-3, -8],
    [-8, -3],
    [-8, -8]
], dtype=float)
```

```
Y_raw = np.array([
```

```

[-3],
[-8],
[-8],
[-3]
], dtype=float)

```

```

def normalize(x, xmin, xmax):
    return (x - xmin) / (xmax - xmin)

```

```

def denormalize(y, xmin, xmax):
    return xmin + y * (xmax - xmin)

```

```

Xn = normalize(X, c0, c1)
Y = normalize(Y_raw, c0, c1)

```

```

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

```

```

def dsigmoid(y):
    return y * (1.0 - y)

```

```

class MLP221:

```

```

    def __init__(self, lr=0.5, seed=7):
        rng = np.random.RandomState(seed)

```

```

        self.W1 = rng.uniform(-0.1, 0.1, (2, 2))
        self.T1 = rng.uniform(-0.1, 0.1, (1, 2))
        self.W2 = rng.uniform(-0.1, 0.1, (2, 1))
        self.T2 = rng.uniform(-0.1, 0.1, (1, 1))
        self.lr = lr

```

```

    def forward(self, x):
        x = x.reshape(1, -1)
        y1 = sigmoid(x @ self.W1 - self.T1)
        y2 = sigmoid(y1 @ self.W2 - self.T2)
        return y1, y2

```

```

    def train_step(self, x, e):
        x = x.reshape(1, -1)
        e = e.reshape(1, -1)
        y1, y2 = self.forward(x)
        delta2 = (y2 - e) * dsigmoid(y2)
        delta1 = (delta2 @ self.W2.T) * dsigmoid(y1)
        self.W2 -= self.lr * (y1.T @ delta2)
        self.T2 += self.lr * delta2
        self.W1 -= self.lr * (x.T @ delta1)
        self.T1 += self.lr * delta1
        return 0.5 * np.sum((y2 - e) ** 2)

```

```

    def predict(self, x):
        return self.forward(x)[1][0, 0]
    def fit(self, X, Y, Ee=0.01, max_epochs=20000, seed=42):
        rng = np.random.RandomState(seed)

```

```

        for epoch in range(1, max_epochs + 1):

```

```

            idx = rng.permutation(len(X))
            Es = 0

```

```

        for i in idx:
            Es += self.train_step(X[i], Y[i])

        if Es <= Ee:
            return epoch, Es

    return max_epochs, Es

class SingleLayerPerceptron:

    def __init__(self, lr=0.5, seed=7):
        rng = np.random.RandomState(seed)

        self.W = rng.uniform(-0.1, 0.1, (2, 1))
        self.T = rng.uniform(-0.1, 0.1, (1, 1))

        self.lr = lr

    def forward(self, x):
        x = x.reshape(1, -1)
        return sigmoid(x @ self.W - self.T)

    def train_step(self, x, e):
        x = x.reshape(1, -1)
        e = e.reshape(1, -1)

        y = self.forward(x)

        delta = (y - e) * dsigmoid(y)

        self.W -= self.lr * (x.T @ delta)
        self.T += self.lr * delta

        return 0.5 * np.sum((y - e) ** 2)

    def predict(self, x):
        return self.forward(x)[0, 0]

    def fit(self, X, Y, Ee=0.01, max_epochs=20000, seed=42):
        rng = np.random.RandomState(seed)

        for epoch in range(1, max_epochs + 1):

            idx = rng.permutation(len(X))
            Es = 0

            for i in idx:
                Es += self.train_step(X[i], Y[i])

            if Es <= Ee:
                return epoch, Es

        return max_epochs, Es

def accuracy(model, X, Y):
    correct = 0

    for x, y in zip(X, Y):

        prediction = model.predict(x)

        predicted_class = int(prediction >= 0.5)

```

```

        correct_class = int(y[0])

        if predicted_class == correct_class:
            correct += 1

    return correct / len(X)

def show_results(model, name):

    print()
    print("=" * 60)
    print(name)
    print("=" * 60)

    print(
        f'{A':>8}"
        f'{B':>8}"
        f'{Ожидается':>15}"
        f'{Выход':>15}"
        f'{Класс':>12}"
    )

    print("-" * 60)

    for x_raw, x_norm, y_raw in zip(X, Xn, Y_raw):

        y_norm = model.predict(x_norm)

        y_hat = denormalize(
            y_norm,
            c0,
            c1
        )

        nearest = c0 if y_norm < 0.5 else c1

        print(
            f'{x_raw[0]:>8.0f}"
            f'{x_raw[1]:>8.0f}"
            f'{y_raw[0]:>15.0f}"
            f'{y_hat:>15.4f}"
            f'{nearest:>12}"
        )

def run_inference(model, A, B, name):

    x = normalize(
        np.array([A, B], dtype=float),
        c0,
        c1
    )

    y_norm = model.predict(x)

    y_hat = denormalize(
        y_norm,
        c0,
        c1
    )

    nearest = c0 if y_norm < 0.5 else c1

```

```

print(
    f"[{name}] "
    f"A={A}, B={B} -> "
    f" $\hat{y}={y\_hat:.3f}$  -> "
    f"ближе к классу {nearest}"
)

if __name__ == "__main__":

    print("=" * 60)
    print("ЛАБОРАТОРНАЯ РАБОТА №1")
    print("Вариант 10: c0 = -3, c1 = -8")
    print("=" * 60)

    print("""
Таблица истинности:

A    B    A XOR B
-3   -3    -3
-3   -8    -8
-8   -3    -8
-8   -8    -3
""")

    mlp = MLP221()
    mlp_epochs, mlp_Es = mlp.fit(Xn, Y)

    slp = SingleLayerPerceptron()
    slp_epochs, slp_Es = slp.fit(Xn, Y)

    mlp_acc = accuracy(mlp, Xn, Y)
    slp_acc = accuracy(slp, Xn, Y)

    print("=" * 60)
    print("РЕЗУЛЬТАТЫ ОБУЧЕНИЯ")
    print("=" * 60)

    print(
        f"\nMLP 2-2-1:"
        f"\nКоличество эпох: {mlp_epochs}"
        f"\nСуммарная ошибка Es: {mlp_Es:.5f}"
        f"\nAccuracy: {mlp_acc:.2f}"
    )

    print(
        f"\nОднослойный персептрон:"
        f"\nКоличество эпох: {slp_epochs}"
        f"\nСуммарная ошибка Es: {slp_Es:.5f}"
        f"\nAccuracy: {slp_acc:.2f}"
    )

    show_results(
        mlp,
        "MLP 2-2-1"
    )

    show_results(
        slp,
        "ОДНОСЛОЙНЫЙ ПЕРСЕПТРОН"
    )

```

```

print()
print("=" * 60)
print("РЕЖИМ ФУНКЦИОНИРОВАНИЯ")
print("=" * 60)

print("Введите два числа A и B из диапазона [-10; 10].")
print("Например: -3 -8")
print("Чтобы выйти, нажмите Enter.")

while True:

    raw = input(
        "\nВведите A B (пусто — выход): "
    ).strip()

    if not raw:
        print("Программа завершена.")
        break

    try:

        A, B = map(float, raw.split())

        if not (-10 <= A <= 10 and -10 <= B <= 10):
            print("Числа должны быть в диапазоне [-10; 10].")
            continue

        run_inference(
            mlp,
            A,
            B,
            "MLP"
        )

        run_inference(
            slp,
            A,
            B,
            "SLP"
        )

    except ValueError:
        print("Введите два числа через пробел.")

```

Результат тестирования:

=====

ЛАБОРАТОРНАЯ РАБОТА №1

Вариант 10: c0 = -3, c1 = -8

=====

Таблица истинности:

A	B	A XOR B
---	---	---------

-3 -3 -3
-3 -8 -8
-8 -3 -8
-8 -8 -3

=====

РЕЗУЛЬТАТЫ ОБУЧЕНИЯ

=====

MLP 2-2-1:

Количество эпох: 3062

Суммарная ошибка Es: 0.01000

Ассурасу: 1.00

Однослойный персептрон:

Количество эпох: 20000

Суммарная ошибка Es: 0.53145

Ассурасу: 0.75

=====

MLP 2-2-1

=====

A	B	Ожидается	Выход	Класс
-3	-3	-3	-3.2797	-3
-3	-8	-8	-7.6668	-8
-8	-3	-8	-7.6669	-8
-8	-8	-3	-3.4422	-3

=====

ОДНОСЛОЙНЫЙ ПЕРСЕПТРОН

=====

A	B	Ожидается	Выход	Класс
-3	-3	-3	-5.5392	-8
-3	-8	-8	-5.5191	-8
-8	-3	-8	-5.5024	-8

-8 -8 -3 -5.4823 -3

=====

РЕЖИМ ФУНКЦИОНИРОВАНИЯ

=====

Введите два числа A и B из диапазона [-10; 10].

Например: -3 -8

Чтобы выйти, нажмите Enter.

Введите A B (пусто — выход): -3 -8

[MLP] A=-3.0, B=-8.0 -> $\hat{y}=-7.667$ -> ближе к классу -8

[SLP] A=-3.0, B=-8.0 -> $\hat{y}=-5.519$ -> ближе к классу -8

Введите A B (пусто — выход): -3 -3

[MLP] A=-3.0, B=-3.0 -> $\hat{y}=-3.280$ -> ближе к классу -3

[SLP] A=-3.0, B=-3.0 -> $\hat{y}=-5.539$ -> ближе к классу -8

Введите A B (пусто — выход): -5 -5

[MLP] A=-5.0, B=-5.0 -> $\hat{y}=-7.725$ -> ближе к классу -8

[SLP] A=-5.0, B=-5.0 -> $\hat{y}=-5.516$ -> ближе к классу -8

Введите A B (пусто — выход): 5 5

[MLP] A=5.0, B=5.0 -> $\hat{y}=-3.136$ -> ближе к классу -3

[SLP] A=5.0, B=5.0 -> $\hat{y}=-5.630$ -> ближе к классу -8

Вывод: Прделав данную лабораторную работу я реализовал многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10, соответствующими логическим классам 0 и 1. Провел сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении однослойным персептроном, показав принципиальную линейную неразделимость задачи XOR.